

Telecom Smart Plan Recommender (OOPS)

Background

ConnectTel offers many plans with different **validities** (28/30/56/84/90/365 days), quotas, and OTT bundles. Customers often choose poorly and get bill shocks. You must design an **OOP-based** system that:

1. Models plans and benefits,
2. Computes a customer's **30-day** cost under each plan,
3. Checks OTT requirements, and
4. Recommends the best plan.

Think through the **logic first** (normalization across different validities) before coding.

Plans (authoritative business data)

All costs in ₹. "GB/day" resets daily within the plan's validity. Where a quota is "total GB," it is for the **entire validity**.

1. **Basic Lite — ₹249, 28 days**
 - Data: **1 GB/day** (overage: ₹0.70/10 MB)
 - Voice: **100 mins included**, then ₹0.75/min
 - SMS: ₹0.20/SMS
 - OTT: none
2. **Saver 30 — ₹499, 30 days**
 - Data: **1.5 GB/day** (overage: ₹0.70/10 MB)
 - Voice: **300 mins included**, then ₹0.75/min
 - SMS: **100 free**, then ₹0.2/SMS
 - OTT: **Hotstar**
3. **Unlimited Talk 30 — ₹650, 30 days**
 - Data: **5 GB total** included (overage: ₹0.70/10 MB)
 - Voice: **Unlimited**
 - SMS: **Unlimited**
 - OTT: **Spotify**
4. **Data Max 20 — ₹749, 20 days**
 - Data: **Unlimited**
 - Voice: **100 mins included**, then ₹0.75/min

- SMS: **Unlimited**
 - OTT: **Hotstar**
5. **Student Stream 56 — ₹435, 30 days**
- Data: **2 GB/day** (overage: ₹0.70/10 MB)
 - Voice: **300 mins included**, then ₹0.75/min
 - SMS: **200 free**, then ₹0.20/SMS
 - OTT: **Spotify**
6. **Family Share 30 — ₹500, 28 days**
- Data: **50 GB total** for validity (overage: ₹0.70/10 MB)
 - Voice: **1000 mins included**, then ₹0.6/min
 - SMS: **500 free**, then ₹0.20/SMS
 - OTT: **Amazon Prime**
7. **Data Max Plus 30 — ₹1499, 30 days**
- Data: **Unlimited**
 - Voice: **300 mins included**, then ₹0.75/min
 - SMS: **200 free**, then ₹0.20/SMS
 - OTT: **Amazon Prime + Hotstar**
8. **Premium Ultra 30 — ₹2999, 30 days**
- Data/Voice/SMS: **Unlimited**
 - OTT: **Netflix + Amazon Prime + Hotstar + Spotify**

Inputs (the customer's 30-day need)

- Voice minutes (e.g., 650)
- SMS count (e.g., 300)
- Data usage in MB (e.g., 8×1024 MB)
- OTT requirements (e.g., needs Netflix? Prime? Hotstar? Spotify?)

Normalization

Mobile/data plans may have different validity periods (10 days, 28 days, 30 days). To compare them fairly, we imagine all plans as if they lasted exactly 30 days.

This normalization ensures every plan is evaluated against the same **30-day** usage window.

Assumption: All inputs describe **one 30-day period**. You do not “buy multiple packs” inside 30 days—just normalize one plan to 30 days as above.

i.e we are **not buying multiple packs** in 30 days. We just take **one plan** and **resize** it to a 30-day window for a fair comparison.

What to Build

1. Domain Model (OOP mandatory)

- Plan (abstract/base) → 8 concrete subclasses with their pricing rules & OTT bundle flags.
- Usage (minutes, sms, dataMB).
- OTTRequirement (booleans for Netflix/Prime/Hotstar/Spotify).
- PlanQuote (planName, totalCost, short breakdown).

2. Pricing

- Each plan computes a PlanQuote price(Usage) using the **normalization rules** above.
- Overages kick in after pro-rated included quotas.

3. Eligibility & Recommendation

- Filter by plans that **meet OTT requirements**.
- From those, pick the **lowest total cost** (ties: choose the plan with more OTTs or simpler pricing—your call, but document it).

4. Output

- Print per-plan **breakdown** (rental_30d, data overage, voice extra, sms extra, total).
- Print the **recommended plan** with reason (cheapest and meets OTT).

Language-Specific Notes

For Java (mandatory OOP)

- **Baseline:** classes, inheritance, encapsulation, overriding.
- **Optional power-ups (if comfortable):**
 - **Collections** (List, Map) for plan lists & filtering,
 - **Streams & lambdas** for concise pipelines,
 - **Generics** where useful,
 - **enum** for OTT apps,
 - **BigDecimal** for precise currency (optional).
- Do **not** rely on exceptions for control flow.

For Python (mandatory OOP)

- **Baseline:** classes/objects, methods, simple lists/loops, no external libs.
- **Optional power-ups (if comfortable):**
 - **dataclasses** for tidy models,
 - **typing** (List, Optional, etc.) for clarity,
 - **enum** for OTT apps,
 - **Comprehensions** and `min(..., key=...)` for clean selection,
 - **abc** for abstract base class feel.

Guided Hints

- Put **all pricing math inside each plan** class.
- Keep a small **PlanOptimizer** that filters by OTT and picks the cheapest.
- Write a **helper** to compute **pro-rated rental and quotas** so the math is consistent across plans.
- Make it easy to **add a 9th plan** without changing existing logic (open/closed principle).

